

6. Les attributs

6.1. Définition d'un attribut

1. Sens simple

Un **attribut** est une **information stockée dans un objet**.

Exemple :

Une classe `Voiture` peut avoir des attributs :

`marque = "Toyota"`

`couleur = "rouge"`

`vitesse = 0`

Ces attributs décrivent **l'état de l'objet**.

2. Idée clé

Un attribut répond à la question :

Qu'est-ce que l'objet possède comme information ?

Exemples :

`Utilisateur`

- `nom`
- `email`
- `motDePasse`

`Produit`

- `prix`
- `nom`
- `quantité`

3. Exemple en code

```
class Voiture {  
    constructor(marque, couleur) {  
        this.marque = marque;  
        this.couleur = couleur;  
    }  
}  
  
const voiture1 = new Voiture("Toyota", "rouge");  
  
console.log(voiture1.marque); // Toyota  
console.log(voiture1.couleur); // rouge
```

Ici :

marque

couleur

sont des **attributs**.

4. À retenir

Un **attribut** = une **donnée** qui appartient à un objet.

Formule simple :

Objet = attributs + méthodes

Exemple :

Voiture = marque + couleur + rouler()

Les **attributs décrivent** l'objet.
Les **méthodes font agir** l'objet.

6.2. Attribut d'instance

1. Définition simple

Un **attribut d'instance** est une **donnée propre à un objet précis**.

Chaque objet peut avoir **ses propres valeurs**.

2. Idée clé

Une **classe** sert de modèle.

Une **instance** est un objet créé à partir de cette classe.

Un **attribut d'instance** appartient à **une instance particulière**.

Exemple :

```
class Voiture {  
    constructor(marque, couleur) {  
        this.marque = marque;  
        this.couleur = couleur;  
    }  
}
```

Ici :

`this.marque`

`this.couleur`

sont des **attributs d'instance**.

3. Exemple concret

```
const voiture1 = new Voiture("Toyota", "rouge");
```

```
const voiture2 = new Voiture("BMW", "noire");

console.log(voiture1.marque); // Toyota
console.log(voiture2.marque); // BMW
```

Chaque voiture a **sa propre marque** et **sa propre couleur**.

4. À retenir

Un **attribut d'instance** = une **information propre à chaque objet**.

Formule simple :

Même classe, objets différents, valeurs différentes.

Exemple :

```
voiture1.couleur = "rouge"
```

```
voiture2.couleur = "noire"
```

Donc : un attribut d'instance décrit **l'état personnel d'un objet**. 🚗

6.3. Attribut de classe

1. Définition simple

Un **attribut de classe** est une **donnée partagée par tous les objets** d'une même classe.

Il appartient à la **classe**, pas à un objet précis.

2. Idée clé 🧠

Un **attribut d'instance** change selon chaque objet.

Un **attribut de classe** est commun à tous.

Exemple :

```
class Voiture {  
    static nombreRoues = 4;  
  
    constructor(marque) {  
        this.marque = marque;  
    }  
}
```

Ici :

`Voiture.nombreRoues`

est un **attribut de classe**.

3. Exemple concret

```
const voiture1 = new Voiture("Toyota");  
const voiture2 = new Voiture("BMW");
```

```
console.log(Voiture.nombreRoues); // 4  
console.log(voiture1.marque);      // Toyota  
console.log(voiture2.marque);      // BMW
```

`nombreRoues` est partagé par toutes les voitures.

`marque` est propre à chaque voiture.

4. À retenir

Un **attribut de classe** = une **information commune à tous les objets**.

Formule simple :

Attribut de classe = valeur partagée par la classe entière.

Exemple :

```
Voiture.nombreRoues = 4
```

Toutes les voitures ont cette même information. 🚗

6.4. Attribut public

1. Définition simple

Un **attribut public** est un attribut **accessible directement depuis l'extérieur** de l'objet.

On peut le **lire** et souvent le **modifier** directement.

2. Idée clé 🧠

Un attribut public est comme une information **visible** de l'objet.

Exemple :

```
class Utilisateur {  
    constructor(nom) {  
        this.nom = nom; // attribut public  
    }  
}
```

Ici :

```
this.nom
```

est un **attribut public**.

3. Exemple concret

```
const user = new Utilisateur("Awa");
```

```
console.log(user.nom); // Awa
```

```
user.nom = "Fatou";
```

```
console.log(user.nom); // Fatou
```

On peut accéder à nom directement avec :

```
user.nom
```

4. À retenir

Un **attribut public** = une **donnée visible et accessible directement**.

Formule simple :

public = accessible de l'extérieur

⚠ Attention : trop d'attributs publics peuvent rendre l'objet moins sécurisé ou moins contrôlé.

6.5. Attribut privé

1. Définition simple

Un **attribut privé** est une donnée **cachée à l'extérieur** de l'objet.

On ne peut pas y accéder directement depuis dehors.

2. Idée clé 🧠

Un attribut privé sert à **protéger une information importante**.

Exemple en JavaScript :

```
class CompteBancaire {
```

```
#solde = 0; // attribut privé  
}
```

Ici :

#solde

est un **attribut privé**.

3. Exemple concret

```
class CompteBancaire {  
    #solde = 1000;  
  
    afficherSolde() {  
        return this.#solde;  
    }  
}
```

```
const compte = new CompteBancaire();
```

```
console.log(compte.afficherSolde()); // 1000
```

Mais ceci est interdit :

```
console.log(compte.#solde); // erreur
```

4. À retenir

Un **attribut privé** = une **donnée protégée** dans l'objet.

Formule simple :

privé = accessible seulement à l'intérieur de la classe

Il permet de mieux contrôler les données sensibles. 🔒

6.6. Attribut protégé

1. Définition simple

Un **attribut protégé** est une donnée accessible :

- dans la **classe elle-même**
- dans ses **classes enfants**

Mais il n'est pas censé être utilisé directement depuis l'extérieur.

2. Idée clé 🧠

Un attribut protégé est entre :

public → visible partout

privé → visible seulement dans la classe

protégé → visible dans la classe + ses enfants

3. Exemple simple

En POO, on le représente souvent avec `_` :

```
class Animal {  
    constructor(nom) {  
        this._nom = nom; // attribut protégé par  
        convention  
    }  
}
```

```
class Chien extends Animal {  
    afficherNom() {  
        return this._nom;  
    }  
}
```

Ici :

`_nom`

est considéré comme un **attribut protégé**.

4. À retenir

Un **attribut protégé** = une donnée réservée à la **classe** et à ses **classes enfants**.

Formule simple :

protégé = utilisable dans la famille de classes 👨‍👩‍👧‍👦

Il sert à partager une information avec les classes enfants sans l'exposer complètement.

6.7. Valeur par défaut d'un attribut

1. Définition simple

Une **valeur par défaut** est la **valeur donnée automatiquement** à un attribut quand aucune valeur n'est fournie.

2. Idée clé 🧠

Elle permet à un objet d'avoir un **état initial correct** dès sa création.

Exemple :

```
class Compte {
```

```
    constructor(nom) {  
        this.nom = nom;  
        this.solde = 0; // valeur par défaut  
    }  
}
```

Ici :

```
solde = 0
```

est une **valeur par défaut**.

3. Exemple concret

```
const compte1 = new Compte("Awa");
```

```
console.log(compte1.nom);    // Awa
```

```
console.log(compte1.solde);  // 0
```

Même si on ne donne pas de solde, l'objet commence avec :

```
solde = 0
```

4. À retenir

Une **valeur par défaut** = une **valeur initiale automatique**.

Formule simple :

pas de valeur donnée → valeur par défaut utilisée

Elle évite les attributs vides ou incohérents. ☒

6.8. Initialisation des attributs

1. Définition simple

L'**initialisation des attributs** consiste à donner des **valeurs de départ** aux attributs d'un objet.

Cela se fait souvent au moment de la **création de l'objet**.

2. Idée clé 🧠

Quand on crée un objet, on prépare son **état initial**.

Exemple :

```
class Utilisateur {  
    constructor(nom, email) {  
        this.nom = nom;  
        this.email = email;  
        this.actif = true;  
    }  
}
```

Ici, les attributs sont initialisés :

```
nom  
email  
actif
```

3. Exemple concret

```
const user1 = new Utilisateur("Awa",  
    "awa@mail.com");  
  
console.log(user1.nom);    // Awa  
console.log(user1.email);  // awa@mail.com
```

```
console.log(user1.actif); // true
```

L'objet reçoit ses premières valeurs dès sa création.

4. À retenir

Initialiser un attribut = lui donner une **valeur de départ**.

Formule simple :

création de l'objet → **valeurs initiales des attributs**

Sans initialisation, l'objet peut être incomplet ou incohérent. 

6.9. Validation des attributs

1. Définition simple

La **validation des attributs** consiste à vérifier qu'une valeur est **correcte** avant de la stocker dans un objet.

2. Idée clé

Un objet ne doit pas accepter n'importe quelle valeur.

Exemple :

```
age = -5 // mauvais
```

```
age = 25 // correct
```

Un âge négatif n'a pas de sens.

3. Exemple simple

```
class Utilisateur {  
    constructor(nom, age) {  
        this.nom = nom;
```

```
    if (age >= 0) {  
        this.age = age;  
    } else {  
        this.age = 0;  
    }  
}  
  
const user = new Utilisateur("Awa", -5);  
  
console.log(user.age); // 0
```

Ici, l'attribut age est contrôlé avant d'être accepté.

4. À retenir

Valider un attribut = vérifier que sa valeur est **logique, autorisée et sûre**.

Formule simple :

valeur reçue → vérification → stockage

La validation protège l'objet contre les données incorrectes. 

6.10. Attributs calculés

1. Définition simple

Un **attribut calculé** est une valeur obtenue à partir d'**autres attributs**.

Il n'est pas toujours stocké directement : il peut être **calculé quand on en a besoin**.

2. Idée clé 🧠

Un attribut calculé répond à une logique :

valeur finale = calcul à partir des données de l'objet

Exemple :

```
prixTotal = prixUnitaire * quantite
```

3. Exemple simple

```
class Produit {  
    constructor(nom, prixUnitaire, quantite) {  
        this.nom = nom;  
        this.prixUnitaire = prixUnitaire;  
        this.quantite = quantite;  
    }  
  
    get prixTotal() {  
        return this.prixUnitaire * this.quantite;  
    }  
}  
  
const produit = new Produit("Riz", 5000, 3);  
  
console.log(produit.prixTotal); // 15000
```

Ici :

prixTotal

est un **attribut calculé**.

4. À retenir

Un **attribut calculé** = une **valeur déduite d'autres attributs**.

Formule simple :

attributs existants → **calcul** → **nouvelle valeur**

Il évite de stocker une donnée qu'on peut retrouver automatiquement.



6.11. Attributs constants

1. Définition simple

Un **attribut constant** est une donnée qui **ne doit pas changer** après sa définition.

Il représente une valeur **fixe**.

2. Idée clé

On l'utilise pour stocker une information stable.

Exemples :

TAUX_TVA = 18

NOMBRE_MOIS_ANNEE = 12

PI = 3.14

Ces valeurs doivent rester les mêmes.

3. Exemple simple

```
class Produit {  
    static TAUX_TVA = 0.18;  
  
    constructor(prix) {  
        this.prix = prix;  
    }  
  
    prixAvecTVA() {  
        return this.prix + this.prix *  
Produit.TAUX_TVA;  
    }  
}  
  
const produit = new Produit(1000);  
  
console.log(produit.prixAvecTVA()); // 1180
```

Ici :

TAUX_TVA

est un **attribut constant**.

4. À retenir

Un **attribut constant** = une **valeur fixe** utilisée par la classe ou l'objet.

Formule simple :

constant = valeur qui ne change pas

Il sert à éviter les valeurs écrites partout dans le code. 

6.12. Bonnes pratiques de nommage des attributs

1. Définition simple

Le **nommage des attributs** consiste à choisir des noms **clairs, précis et compréhensibles** pour les données d'un objet.

Un bon nom permet de comprendre rapidement le rôle de l'attribut.

2. Idée clé

Un attribut doit dire **ce qu'il contient**.

Mauvais :

```
x = "Awa"
```

```
p = 5000
```

Bon :

```
nom = "Awa"
```

```
prix = 5000
```

3. Exemple simple

```
class Produit {  
    constructor(nom, prixUnitaire, quantiteStock)  
    {  
        this.nom = nom;  
        this.prixUnitaire = prixUnitaire;  
        this.quantiteStock = quantiteStock;  
    }  
}
```

```
}
```

Ici, les noms sont clairs :

```
nom
```

```
prixUnitaire
```

```
quantiteStock
```

On comprend directement ce que chaque attribut représente.

4. À retenir

Un bon nom d'attribut doit être :

- **clair**
- **précis**
- **cohérent**
- **lisible**

Formule simple :

bon nom = rôle de l'attribut compris immédiatement

Exemple :

```
dateNaissance
```

```
emailUtilisateur
```

```
prixTotal
```

Évite les noms vagues comme x, data, info. ☒